

signals of the design to be observed for each cycle during silicon execution. To achieve this, relevant signals are routed to an observation point, which can be either an output pin or a designated section of the memory (referred to as trace buffer).

In addition to these two architectures, there are also instrumentations to transport internal register values off-chip, quickly access large memory arrays and so on. For example, in recent SoC designs, data transport mechanisms may re-purpose some of the communication mechanisms already present in the system, like USB ports. This requires a thorough understanding of both functionality and validation use-cases to ensure that these do not interfere when using the same interface.

Finally, there is instrumentation to provide controllability of execution, for example, by overriding system configuration, updating microcode on-the-fly during execution and so on.

There has recently been significant research on improving post-silicon observability through disciplined DfD architecture. Debug software is another crucial component of post-silicon validation readiness. It includes any software tool and infrastructure necessary to enable running post-silicon tests and facilitating debug, triage or validation of different coverage goals.

Debug software can be categorised into the following classes:

Instrumented system software. Post-silicon validation, particularly for hardware logic and compatibility validation, requires running long and complicated tests, identifying complex corner cases and root causing errors excited by subtle hardware/software coordination. To achieve this, one needs to run an application software stack on the target system. Doing this by executing an application on top of an off-the shelf operating system is difficult. Modern operating systems (like Linux, Windows, Android and MacOS) are highly optimised for performance and power consumption, and are significantly complex.

To enable debug of underlying hardware issues, one needs a highly-customised system software, with a reduced set of bells and whistles

while including a number of hooks or instrumentations to facilitate debug, observability and control. For example, one may want to trace the sequence of branches taken by an application in order to excite a specific hardware problem. To achieve this, often specialised operating systems are implemented that are targeted for silicon debug. Such system software may be written by silicon debug teams from scratch or by significantly modifying off-the-shelf implementations.

Tracing, triggers and configurations. Some customised software tools are also developed for controlling, querying and configuring the internal state of the silicon. In particular, there are tools to query or configure specific hardware registers, setting triggers for tracing and so on. For example, one may wish to trace specific signal S only when internal register R contains specific value v. Assuming that both S and R are observable, one needs software tools to query R and configure signal tracing to include S when R contains v.

Transport software. Access software refers to tools that enable transport of data off-chip from silicon. Data can be transferred off-chip either directly through pins, or by using available ports from the platform (USB, PCIe, etc). For example, transporting through the USB port requires instrumentation of the USB driver to interpret and route the debug data while ensuring that USB functionality is not affected during normal execution. This can become highly complex and subtle, particularly in the presence of other features in the SoC, such as power management.

Power management may, in fact, power down the USB controller when the USB port is not being used by the functional activity of the system. The instrumented driver ensures that debug data is still being transported while facilitating the power-down functionality of the hardware to be exercised during silicon validation.

Analysis software. Finally, there are software tools to perform analysis on the transported data. There are tools to aggregate the raw signal or trace data into high-level data structures (for example, interpreting signal

streams from the communication fabric in the SoC as messages or transactions among IPs), comprehending and visualising hardware/software coordinations as well as tools to analyse such traced and observed data for further high-level debug (such as estimating congestion across the communication fabric, traffic patterns during internal transactions and power consumption during system execution).

One critical challenge in developing (and validating) debug software is its tight integration with the target hardware design to be validated. Typically, software development and validation make use of a stable hardware platform, that is, developing application software on top of a general-purpose hardware instruction set architecture such as X86 or ARM. However, debug software is developed for an under-development target platform often with evolving and changing features (like, in response to design or architectural challenges discovered late). This makes debug software design a vexing and complex problem in hardware/software co-design and co-validation.

Indeed, it is not uncommon in post-silicon validation to root cause an observed failure to an issue with the debug software rather than the target system. Developing a streamlined technology for debug software development and validation is a challenging area of research.

Test generation and testing setup design

The central component of silicon debug is the set of tests to run. For validation to be effective, the tests must expose potential vulnerabilities of the design and exercise different corner cases and configurations.

Post-silicon tests. Post-silicon tests can be divided into the following two categories:

Focused (directed) tests. Such tests are carefully crafted by expert test writers to target specific features of the system (like, multiprocessor and chipset protocols, CPU checks for specific register configurations, address decoding and power management features). Developing such tests